

Part A UNIX I/O 基础性能对比实验报告

系统程序设计 Lab 1

2026-05-14

Part A UNIX I/O 基础性能对比实验报告

1. 实验目的

本实验测量 UNIX/Linux 下多种 I/O 接口在顺序读写场景中的性能差异，重点观察系统调用次数、用户态缓冲、内核页缓存以及 `real/user/sys` 时间的影响。实验覆盖 `read()`、`getc()`、`fgetc()`、`fread()`、自实现 `my_fread()`、普通 `write()` 和带 `O_SYNC` 的 `write()`。

2. 实验环境

项目	信息
操作系统	Ubuntu 20.04.6 LTS
内核	Linux rdma221 5.4.0-26-generic #30-Ubuntu SMP Mon Apr 20 16:58:30 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
CPU	Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz, 64 logical CPUs
内存	Mem: 251Gi 110Gi 134Gi 2.0Mi 6.0Gi 138Gi
文件系统	/dev/mapper/ubuntu--vg-ubuntu--lv ext4 1880200276 1225559920 568832300 69% /
磁盘	服务器存在 SSD/NVMe 设备，lsblk 显示 ROTA=0，包括 WDC_WDS100T2G0A- 与 Dell DC NVMe PE8010 RI U.2 960GB

环境信息原始输出保存在 `results/env/` 目录下。实验结果 CSV 保存在 `results/` 目录下。

3. 实验方法

3.1 计时方法

测试程序使用 `clock_gettime(CLOCK_MONOTONIC)` 记录 wall clock time，也就是报告中的 `real_sec`；使用 `getrusage(RUSAGE_SELF)` 记录用户态 CPU 时间 `user_sec` 和内核态 CPU 时间 `sys_sec`。每次测试均输出 CSV：

`method,bufsize,bytes,real_sec,user_sec,sys_sec`

本实验结果为服务器上的一次完整运行。由于 UNIX/Linux 文件 I/O 会受到 page cache、后台写回和系统负载影响，报告重点分析 `BUFSIZE` 的数量级趋势，而不是把某个单点时间视为硬件绝对性能。

3.1.1 实验取舍

这组实验最容易被“小 `BUFSIZE`”拖慢。以 500 MB 文件为例，`bufsize=1` 意味着 5 亿多次系统调用；如果再叠加 `O_SYNC`，实验时间会从分钟级膨胀到不可接受的程度。因此读实验严格使用 500 MB 文件，保证读取接口的比较有足够数据量；`write()` 也保留了一组 500 MB 数据，用来观察系统调用次数下降后的整体趋势；`O_SYNC write` 则使用 64 KB 和 1 MB 两组较小写入量，重点比较相同 `BUFSIZE` 序列下同步写语义带来的额外成本。

这个取舍不是为了回避慢结果，而是为了让实验能够完整覆盖所有 `BUFSIZE`。后面的分析也不会把 1 MB 的 `O_SYNC` 结果和 500 MB 的普通 `write` 做绝对时间比较，而是主要看同一写入量下普通写与同步写的相对差异。

3.2 缓冲区大小

按照实验要求，使用以下 `BUFSIZE`：

```
1, 2, 4, 8, 16, 32, 64, 128, 256, 512,  
1024, 2048, 4096, 8192, 16384, 32768,  
65536, 16777216
```

3.3 读实验

读实验使用 500 MB 测试文件，命令为：

```
./io_bench all_read testfile.bin > ../results/partA_all_read.csv
```

其中 `read`、`fread`、`my_fread` 对所有 `BUFSIZE` 进行测试；`getc` 与 `fgetc` 是逐字符接口，因此各测试一次。

3.4 写实验

普通 `write` 首先进行了 500 MB 完整测试：

```
./io_bench all_write out.bin 524288000 > ../results/partA_all_write.csv
```

该文件中保留的是普通 `write` 的完整结果。由于 `O_SYNC` 在极小 `BUFSIZE` 下会产生大量同步写路径，500 MB 的 `write_sync, bufsize=1` 会带来不可接受的运行时间，因此同步写对比采用 64 KB 和 1 MB 两组较小总写入量，保持相同 `BUFSIZE` 序列：

```
./io_bench write out_1MB.bin 1048576 > ../results/partA_write_1MB.csv  
./io_bench write_sync out_sync_1MB.bin 1048576 > ../results/partA_write_sync_1MB.csv
```

这样设计可以保证 `write` 与 `O_SYNC write` 在相同写入总量下比较，同时避免小缓冲同步写将实验时间放大到数小时。

4. 实验结果

4.1 read / fread / my_fread 趋势

`read()` 在 `bufsize=1` 时 `real time` 为 112.720405 s；当 `BUFSIZE` 增大到 65536 字节时，`real time` 降至 0.065621 s。`fread()` 最优 `real time` 为 0.067050 s，出现在 `BUFSIZE=65536`；`my_fread()` 最优 `real time` 为 0.076267 s，出现在 `BUFSIZE=16384`。

从 `read()` 的 `real/user/sys` 分解可以看到，小 `BUFSIZE` 时 `sys_sec` 占主导。这说明性能瓶颈主要来自频繁进入内核的系统调用开销。`BUFSIZE` 增大，系统调用次数近似按比例下降，`sys_sec` 和 `real time` 也快速下降。到 16 KB 到 64 KB 附近后，继续增大缓冲区收益有限，说明此时已经接近顺序读取和页缓存路径的稳定吞吐区间。

4.2 getc 与 fgetc

`getc()` 与 `fgetc()` 都是逐字符接口，但它们并不等价于每字节调用一次 `read()`。标准 I/O 库内部维护用户态缓冲区，底层会批量读入 500 MB 文件均约为 1.36 s，远快于 `read(bufsize=1)` 的 112.72 s。

本次实验中 `getc()` 和 `fgetc()` 的 `real time` 非常接近，说明在当前编译器、libc 和优化条件下，宏形式或函数形式带来的差异不是 `stdio` 的状态检查和接口调用路径。

4.3 fread 与 my_fread

`fread()` 和 `my_fread()` 都体现了用户态缓冲减少系统调用次数的价值。`my_fread()` 内部使用 `read()` 补充缓冲区，再从用户态缓冲读入。`fread()` 基本一致：小 `BUFSIZE` 下系统调用次数多，性能较差；中等 `BUFSIZE` 后进入平台吞吐上限。

`fread()` 的最优结果略好于 `my_fread()`，原因可能包括：标准库实现经过高度优化；`stdio` 缓冲、内部拷贝和分支处理更成熟；而 `my_fread()` 是教学目的的简化实现，主要用于验证缓冲思想，而非替代 libc。

还有一个细节值得注意：`my_fread(bufsize=1)` 的表现接近逐字节 `read()`，因为外部每次只请求 1 字节，内部缓冲区容量也随测试 `BUFSIZE` 设置为 1 字节，此时无法发挥“先批量读入、再在用户态分发”的优势。这说明缓冲机制本身并不是魔法，关键仍在于4

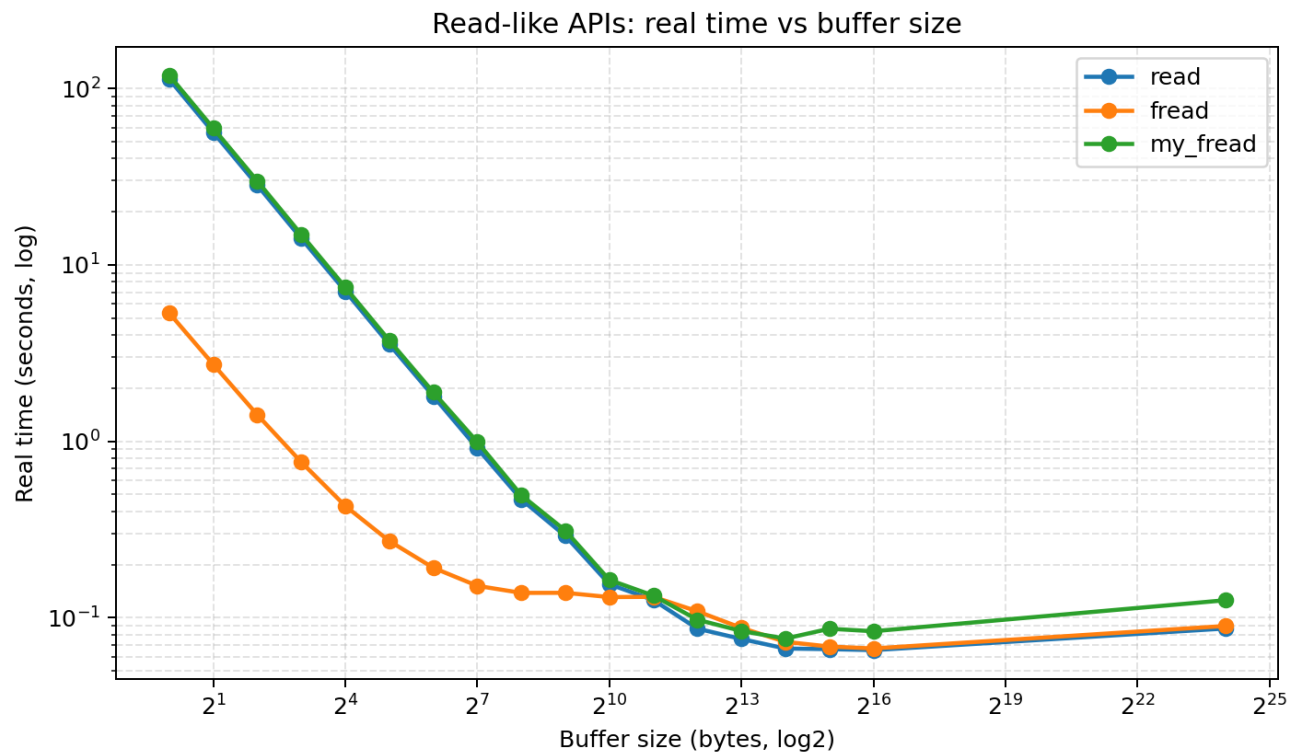


Figure 1: read/fread/my_fread real time

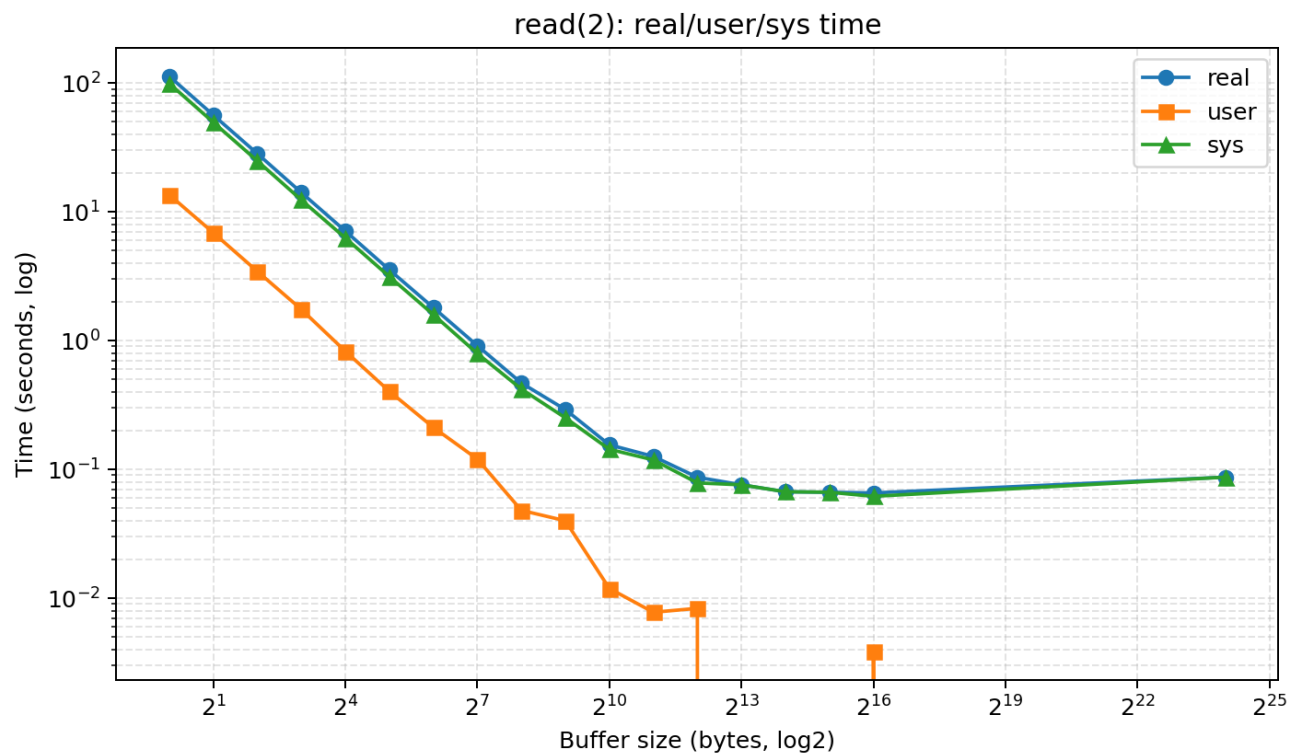


Figure 2: read real/user/sys breakdown

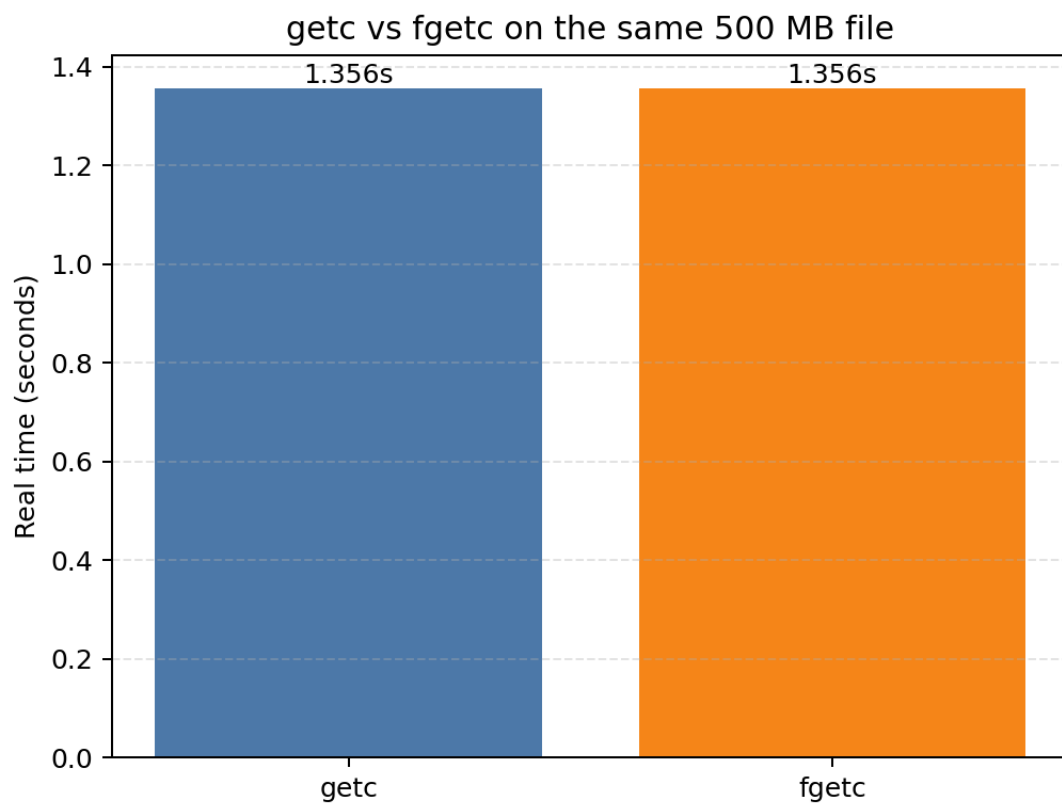


Figure 3: `getc` vs `fgetc`

4.4 普通 write

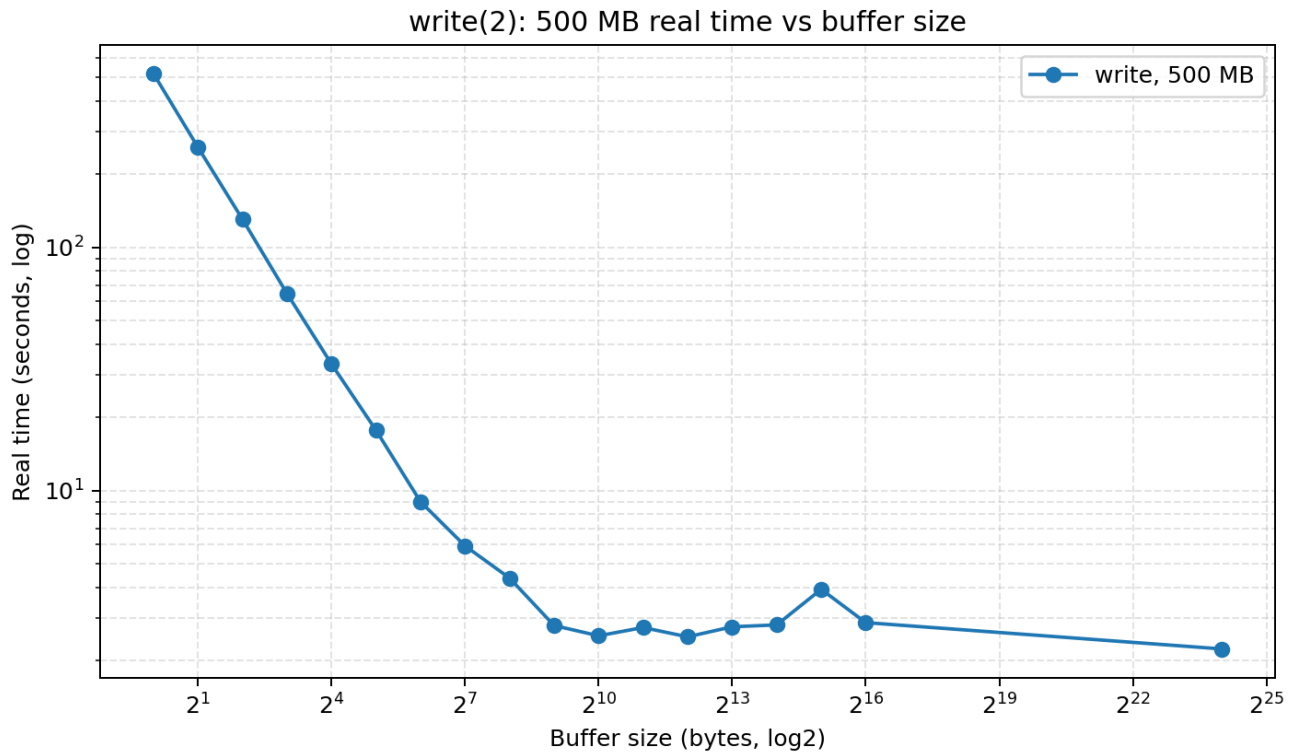


Figure 4: write 500MB

普通 `write()` 的 500 MB 实验中, `bufsize=1` 的 real time 为 519.885968 s; 最优结果为 2.231312 s, 出现在 `BUFSIZE=16777216`。趋势与 `read()` 类似: 极小 `BUFSIZE` 导致系统调用次数巨大, 例如 500 MB / 1 byte 约为 5.24 亿次 `write()` 调用, 因此 `sys_sec` 和 real time 都非常高。

当 `BUFSIZE` 增大到 KB 级以后, 系统调用次数大幅减少, 普通 `write()` 的耗时降到几秒级。普通写不会要求每次调用都同步落盘, real time 主要反映进入页缓存和文件系统路径的成本, 而不是每次真实落盘延迟。

4.5 write 与 O_SYNC write

1 MB 对比实验中, 普通 `write(bufsize=1)` real time 为 1.105486 s, 而 `write_sync(bufsize=1)` real time 为 560.828087 s。`O_SYNC` 要求写操作具备同步语义, 小缓冲区下相当于把大量细碎写入逐次推入更重的同步路径, 因此时间急剧上升。

当 `BUFSIZE` 增大时, `O_SYNC` 的耗时快速下降。`write_sync` 的最优结果为 0.013124 s, 出现在 `BUFSIZE=65536`。这说明 `O_SYNC` 的主要代价不是简单的用户态函数调用, 而是同步提交次数。把多个字节合并进一次较大的 `write()` 可以显著减少同步操作次数。

本次实验中 `write_sync` 的 `sys_sec` 明显小于 `real_sec`, 尤其在小 `BUFSIZE` 下更明显。这说明进程有大量时间并不是在用户态消耗 CPU, 而是在等待同步写路径完成。普通 `write()` 则更多体现系统调用和内核页缓存路径开销。

5. 原理分析与讨论

5.1 系统调用次数

系统调用需要从用户态切换到内核态, 进行参数检查、文件描述符查找、VFS/文件系统路径处理以及数据拷贝等工作。对于 500 MB 文件, `bufsize=1` 会产生数亿次系统调用, 因此时间几乎完全被系统调用开销吞噬。`BUFSIZE` 每翻倍, 理论上的系统调用 real time 也呈现近似减半趋势。

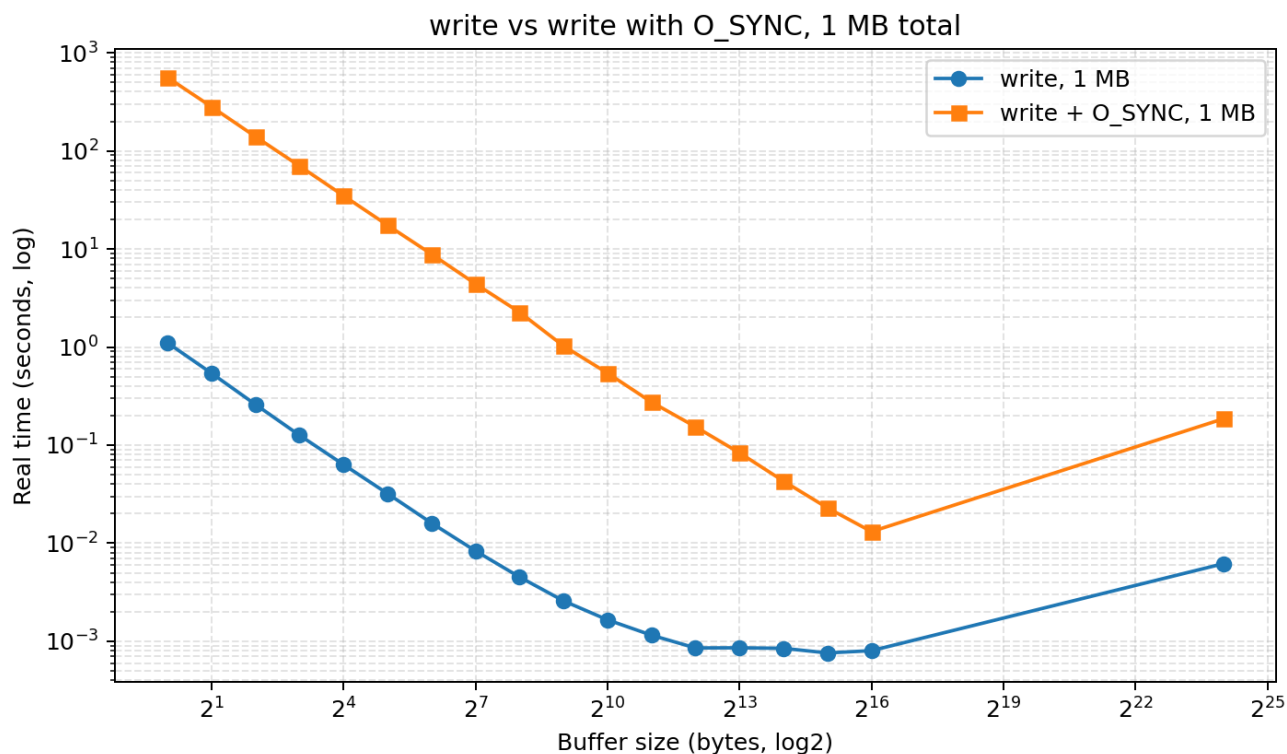


Figure 5: write vs O_SYNC write

5.2 内核页缓存

普通 `read()` 和 `write()` 都会经过内核页缓存。顺序读时，如果数据已经在页缓存中，读取可以主要在内存路径完成；顺序写时，`write()` 通常先把数据写入页缓存并标记脏页，随后由内核异步回写。页缓存使得普通写在大缓冲区下看起来非常快，但这并不

5.3 用户态缓冲

stdio 的 `getc/fgetc/fread` 使用用户态缓冲区，能够减少底层 `read()` 系统调用次数。逐字符的 `getc/fgetc` 虽然 API 上每次返回一个字符，但大多数调用只在用户态缓冲区移动指针，只有缓冲区耗尽时才进入内核补充数据。因此它们明显快于 `read(bufsize=1)`。

5.4 O_SYNC

`O_SYNC` 改变了写入语义：调用返回前需要满足更强的同步要求。小 `BUFSIZE` 下，`O_SYNC` 会把每个小块写入都变成昂贵的同步写。`write_sync(bufsize=1)` 的时间远高于普通 `write(bufsize=1)`，正说明同步写的主导成本在内核和存储路径，而非用户态循环。

5.5 BUFSIZE 的收益递减

从读写实验都可以看到，`BUFSIZE` 从 1 增大到 4 KB、16 KB、64 KB 时收益巨大；但继续增大到 16 MB 后并不一定更快，甚至可能变慢。这是因为系统调用次数已经足够少，新的瓶颈转移到页缓存、内存带宽、文件系统路径、缓存

6. 结论

1. 小 `BUFSIZE` 下，系统调用次数是 UNIX I/O 性能的主要瓶颈。
2. stdio 的用户态缓冲使逐字符 `getc/fgetc` 远快于逐字节 `read()`。
3. `fread()` 与自实现 `my_fread()` 的趋势一致，证明用户态缓冲能显著减少系统调用开销；标准库实现整体更成熟。
4. 普通 `write()` 受益于内核页缓存，大缓冲区下耗时显著下降。
5. `O_SYNC` 会显著放大同步写成本，尤其在极小 `BUFSIZE` 下表现最明显。
6. `BUFSIZE` 并非越大越好，达到 KB 到几十 KB 级别后通常进入收益递减区间。

附录 A: 关键实验数据

A.1 读实验完整数据

method	bufsize	bytes	real_sec	user_sec	sys_sec
read	1	524288000	112.720405	13.555514	99.162456
read	2	524288000	56.389497	6.876000	49.512025
read	4	524288000	28.304440	3.451965	24.851755
read	8	524288000	14.111344	1.747858	12.362964
read	16	524288000	7.093440	0.827666	6.265567
read	32	524288000	3.545144	0.404121	3.140884
read	64	524288000	1.803606	0.211946	1.591594
read	128	524288000	0.914014	0.120238	0.793743
read	256	524288000	0.468600	0.048072	0.420514
read	512	524288000	0.291270	0.039911	0.251353
read	1024	524288000	0.155121	0.011892	0.143216
read	2048	524288000	0.126117	0.007773	0.118344
read	4096	524288000	0.086841	0.008340	0.078501
read	8192	524288000	0.075882	0.000000	0.075882
read	16384	524288000	0.067002	0.000000	0.067002
read	32768	524288000	0.066450	0.000000	0.066441
read	65536	524288000	0.065621	0.003881	0.061739
read	16777216	524288000	0.087001	0.000000	0.087001
getc	1	524288000	1.356019	1.267840	0.088105
fgetc	1	524288000	1.355903	1.295943	0.059892
fread	1	524288000	5.335792	5.267792	0.067823
fread	2	524288000	2.731405	2.651813	0.079467
fread	4	524288000	1.415656	1.355897	0.059727
fread	8	524288000	0.762942	0.692459	0.070424
fread	16	524288000	0.430462	0.383731	0.046728
fread	32	524288000	0.272451	0.184073	0.088375
fread	64	524288000	0.192069	0.108009	0.084059
fread	128	524288000	0.151793	0.063962	0.087830
fread	256	524288000	0.138466	0.043731	0.094674
fread	512	524288000	0.138613	0.024441	0.114172
fread	1024	524288000	0.131075	0.027842	0.103233
fread	2048	524288000	0.131527	0.019922	0.111604
fread	4096	524288000	0.108925	0.000156	0.108768
fread	8192	524288000	0.087679	0.003942	0.083718
fread	16384	524288000	0.073081	0.000183	0.072880
fread	32768	524288000	0.068745	0.003450	0.065295
fread	65536	524288000	0.067050	0.000515	0.066535
fread	16777216	524288000	0.089926	0.000000	0.089927
my_fread	1	524288000	118.695359	19.335005	99.357018
my_fread	2	524288000	59.509369	9.587921	49.919550
my_fread	4	524288000	29.901452	4.644084	25.256383
my_fread	8	524288000	14.871536	2.535842	12.335201
my_fread	16	524288000	7.462318	1.179665	6.282331
my_fread	32	524288000	3.728230	0.592021	3.136102
my_fread	64	524288000	1.897971	0.396321	1.501607
my_fread	128	524288000	0.991997	0.171992	0.819948
my_fread	256	524288000	0.495169	0.107852	0.387298
my_fread	512	524288000	0.312315	0.052046	0.260233
my_fread	1024	524288000	0.163978	0.031999	0.131978
my_fread	2048	524288000	0.133683	0.007618	0.126052
my_fread	4096	524288000	0.097559	0.012253	0.085305

method	bufsize	bytes	real_sec	user_sec	sys_sec
my_fread	8192	524288000	0.084199	0.004031	0.080151
my_fread	16384	524288000	0.076267	0.008049	0.068218
my_fread	32768	524288000	0.086734	0.003789	0.082938
my_fread	65536	524288000	0.083941	0.035988	0.047938
my_fread	16777216	524288000	0.125833	0.036298	0.089535

A.2 500 MB 普通 write 数据

method	bufsize	bytes	real_sec	user_sec	sys_sec
write	1	524288000	519.885968	28.398364	491.344692
write	2	524288000	258.583793	14.599576	243.874502
write	4	524288000	130.718166	7.192194	123.458445
write	8	524288000	64.723741	3.499749	61.135711
write	16	524288000	33.183752	1.852301	31.037260
write	32	524288000	17.716470	0.899073	15.700131
write	64	524288000	8.998366	0.444114	8.042532
write	128	524288000	5.922992	0.194580	4.307318
write	256	524288000	4.366783	0.095779	2.479726
write	512	524288000	2.786438	0.055629	1.350041
write	1024	524288000	2.527278	0.031823	0.868847
write	2048	524288000	2.733176	0.005125	0.650786
write	4096	524288000	2.502849	0.026952	0.494828
write	8192	524288000	2.752775	0.000000	0.514676
write	16384	524288000	2.802269	0.007150	0.525769
write	32768	524288000	3.925510	0.000373	0.486582
write	65536	524288000	2.863585	0.000433	0.503786
write	16777216	524288000	2.231312	0.001910	0.520035

A.3 1 MB 普通 write 数据

method	bufsize	bytes	real_sec	user_sec	sys_sec
write	1	1048576	1.105486	0.066258	1.039201
write	2	1048576	0.540919	0.015797	0.524927
write	4	1048576	0.259634	0.007970	0.251427
write	8	1048576	0.126141	0.004090	0.121824
write	16	1048576	0.063369	0.003959	0.059174
write	32	1048576	0.032080	0.011996	0.019855
write	64	1048576	0.016067	0.000000	0.015845
write	128	1048576	0.008348	0.000000	0.008087
write	256	1048576	0.004527	0.000009	0.004227
write	512	1048576	0.002583	0.000000	0.002363
write	1024	1048576	0.001655	0.000000	0.001396
write	2048	1048576	0.001154	0.000000	0.000929
write	4096	1048576	0.000855	0.000000	0.000635
write	8192	1048576	0.000858	0.000000	0.000597
write	16384	1048576	0.000848	0.000000	0.000571
write	32768	1048576	0.000761	0.000000	0.000549
write	65536	1048576	0.000803	0.000000	0.000550
write	16777216	1048576	0.006187	0.000000	0.006188

A.4 1 MB O_SYNC write 数据

method	bufsize	bytes	real_sec	user_sec	sys_sec
write_sync	1	1048576	560.828087	3.104246	150.485874
write_sync	2	1048576	281.716940	1.394330	76.682559
write_sync	4	1048576	139.638537	0.674138	38.077848
write_sync	8	1048576	70.089779	0.374303	18.888813
write_sync	16	1048576	34.917893	0.172230	9.526610
write_sync	32	1048576	17.469682	0.080044	4.817529
write_sync	64	1048576	8.856736	0.047617	2.405512
write_sync	128	1048576	4.407117	0.021724	1.210233
write_sync	256	1048576	2.265506	0.008040	0.617827
write_sync	512	1048576	1.025391	0.000000	0.317214
write_sync	1024	1048576	0.542591	0.008237	0.155851
write_sync	2048	1048576	0.274431	0.000340	0.086984
write_sync	4096	1048576	0.153762	0.003979	0.044351
write_sync	8192	1048576	0.083410	0.000000	0.025990
write_sync	16384	1048576	0.042934	0.000000	0.014283
write_sync	32768	1048576	0.022896	0.000000	0.008388
write_sync	65536	1048576	0.013124	0.000000	0.005404
write_sync	16777216	1048576	0.185710	0.000000	0.020884